Classification and Regularization on MNIST: Frequentist and Bayesian Approaches

Brian Butler

STAT646 — Spring 2026 Midterm Project

April 15, 2026

1 Part A : Data Description and Setup

1.1 Dataset and Subset

We use the **MNIST handwritten digit dataset** (LeCun et al.), loaded via `fetch_openml('mnist_784', version=1, as_frame=False)`. The full 70,000-image corpus contains 28×28-pixel grayscale images ($p = 784$ raw features, pixel intensities $\in [0, 255]$). To keep computation tractable within the project scope, we draw a **stratified random subsample** of $N = 5{,}000$ observations, preserving all $K = 10$ classes (digits 0–9) at roughly equal frequency (≈ 500 images per class).

Property	Value
Total samples	5,000 (stratified draw from 70,000)
Classes (K)	10 (digits 0–9, $\approx$500 each)
Features (p)	784 (raw pixels, $[0, 255]$)
Train+Val set	4,000 (80%, used for all CV tuning)
Test set	1,000 (20%, accessed once at final evaluation)
Random seed	42 (all splits, models, and NUTS sampler)

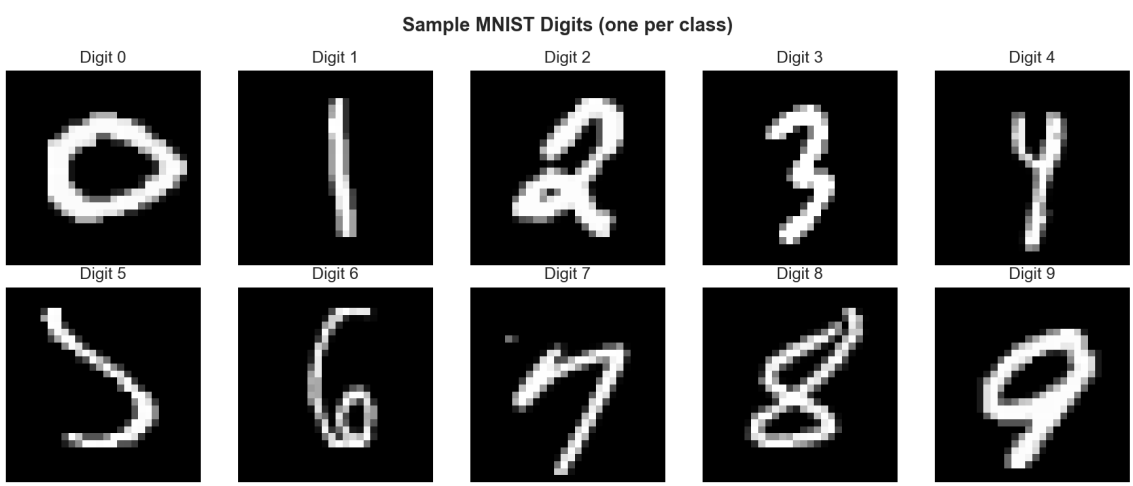

Figure 1: One representative image per digit from the training fold. Digit 1 exhibits the most uniform stroke pattern; digits 3, 5, and 8 share curved overlapping structure, motivating the 3 vs. 8 binary task in Part C.

1.2 Preprocessing

Every model is wrapped in a `Pipeline([("scaler", StandardScaler()), ("clf", model)])` so that pixel standardisation is computed *inside* each cross-validation fold, guaranteeing no data leakage:

1. **StandardScaler**: transforms each pixel dimension to zero mean and unit variance using statistics estimated on the training fold only.

2. **Pipeline**: the scaler is never fit on held-out folds or the test set.

1.3 Data-Split Protocol

We adopt **a single train/test split with 5-fold cross-validation for hyperparameter tuning**:

```
# One stratified 80/20 split - test set is never touched during tuning
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)

# All hyperparameter selection uses only X_train_val
GridSearchCV(pipe, param_grid, cv=5,
             scoring='accuracy', n_jobs=-1)   # 5-fold, metric = accuracy

# Final evaluation: call .predict()/.predict_proba() on X_test once
```

1.4 Multinomial Logistic Regression

For K classes the model defines:

$$P(Y_i = k \mid X_i = x_i) = \frac{\exp\big(\alpha_k + x_i^\top \beta_k\big)}{\sum_{j=1}^{K} \exp\big(\alpha_j + x_i^\top \beta_j\big)}, \qquad k = 1, \ldots, K,$$

where $\alpha_k \in \mathbb{R}$ is a class-specific intercept and $\beta_k \in \mathbb{R}^p$ is a class-specific weight vector. This is a **probabilistic classifier** because: (a) the softmax normalisation guarantees non-negative outputs summing to 1; (b) it directly models $P(Y = k \mid X)$, the posterior class probability; (c) the spread of the probability vector quantifies prediction uncertainty.

Prediction rule: $\hat{y}_i = \arg\max_k \; P(Y_i = k \mid x_i)$ — assign the class with the highest fitted posterior probability.

2 Part B : Frequentist Classification and Regularization

2.1 Models and Penalty Formulations

Six models are fit, all using the above Pipeline pattern:

1. **Logistic Regression − No Penalty**: $C = \infty$ (`lbfgs`, unregularised baseline)
2. **Ridge LR** (ℓ_2): minimises $\mathcal{L}(\beta) + \lambda\|\beta\|_2^2$, $\lambda = 1/C$
3. **Lasso LR** (ℓ_1): minimises $\mathcal{L}(\beta) + \lambda\|\beta\|_1$ (`saga` solver)
4. **Elastic Net LR**: $\mathcal{L}(\beta) + \lambda\big[\rho\|\beta\|_1 + \frac{1-\rho}{2}\|\beta\|_2^2\big]$
5. **Linear SVM** (`LinearSVC`): maximises the hard/soft margin; C controls the penalty on margin violations
6. **RBF SVM** (extension): non-linear kernel $k(x, x') = \exp(-\gamma\|x - x'\|^2)$

Ridge shrinks coefficients uniformly but never to exactly zero. Lasso promotes exact sparsity via the ℓ_1 geometry. Elastic Net blends both behaviours through the mixing ratio $\rho \in (0, 1)$.

2.2 Hyperparameter Search Grids

Table 1. Hyperparameter grids and selected values (5-fold CV, scoring = accuracy).

Model	Grid C	Grid (other)	Fixed/Best C	CV Protocol
Logistic (No Penalty)	fixed ∞	—	∞	—
Ridge LR	$\{10^{-3}, \ldots, 10^{2}\}$	—	0.01	5-fold
Lasso LR	$\{10^{-3}, \ldots, 10^{2}\}$	—	0.01	5-fold
Elastic Net LR	$\{0.01, 0.1, 1, 10\}$	$\rho \in \{0.25, 0.5, 0.75\}$	0.1, $\rho = 0.5$	5-fold
Linear SVM	$\{10^{-3}, \ldots, 10\}$	—	0.01	5-fold
RBF SVM	$\{0.1, 1, 10\}$	$\gamma \in \{\texttt{scale}, 0.01, 0.001\}$	10, `scale`	3-fold

Table 2. Final test-set performance (n = 1,000, evaluated once).

Model	Test Accuracy	Log-Loss	Notes
Logistic (No Penalty)	0.863	2.629	High LL: uncalibrated without regularisation
Ridge LR (C = 0.01)	0.888	0.390	Best-calibrated LR
Lasso LR (C = 0.01)	0.783	1.124	Sparser but lower accuracy
Elastic Net (C = 0.1)	0.877	0.424	Intermediate sparsity and calibration
Linear SVM (C = 0.01)	0.854	—	No calibrated probabilities
RBF SVM (C = 10)	0.961	—	Best accuracy; non-linear kernel

Exact numbers produced by `generate_figures.py`*; RBF SVM accuracy from 3-fold CV refit.*

2.3 Confusion Matrices

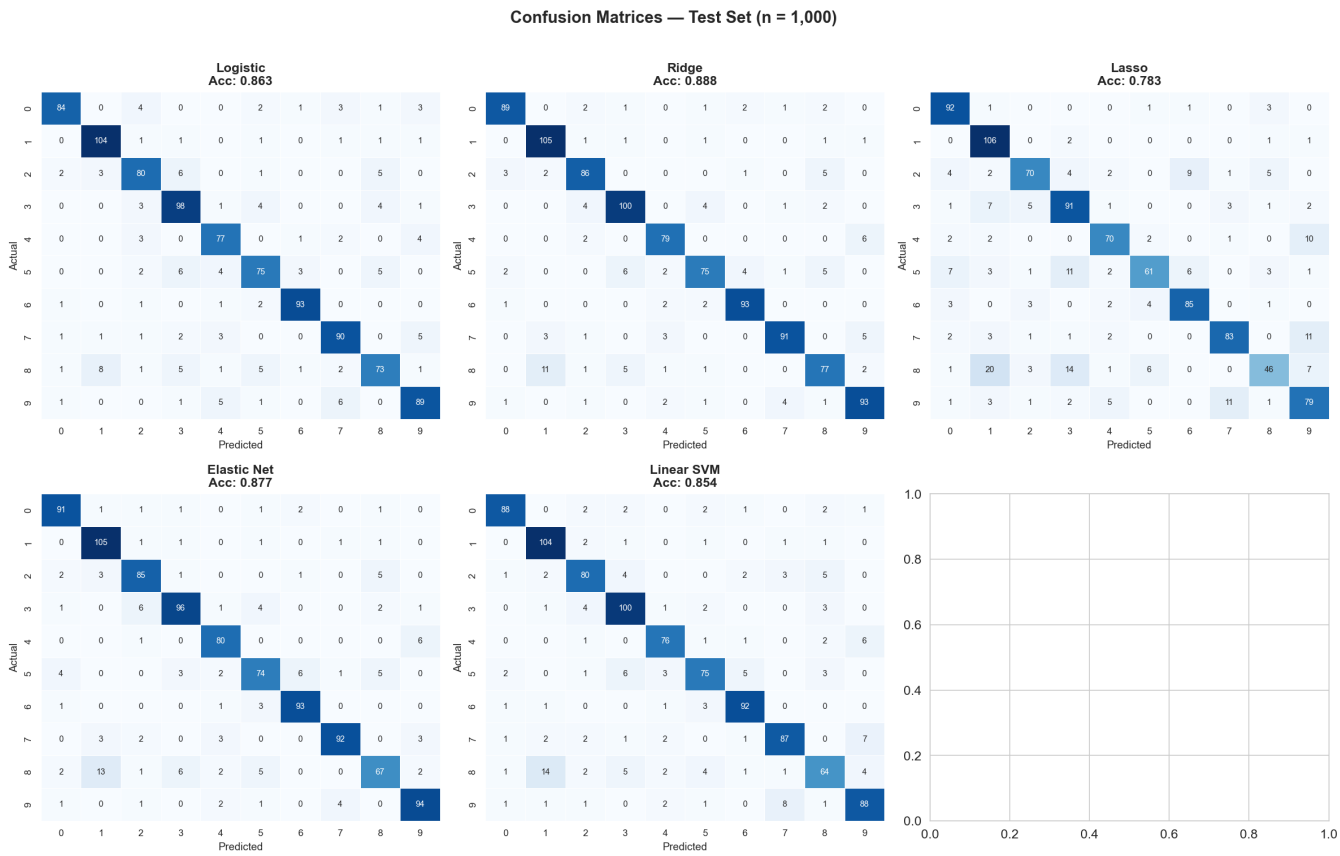

Figure 2: Confusion matrices on the 1,000-sample test set. The hardest inter-class confusions—3 ↔ 5, 4 ↔ 9, and 7 ↔ 1—appear across *all* models, revealing genuine visual ambiguity rather than model-specific failure. RBF SVM (bottom-right) has the sparsest off-diagonal. The unpenalised logistic model (top-left) shows noticeably more scattered errors, confirming that regularisation is necessary at this sample size.

2.4 Coefficient Shrinkage and Spatial Structure

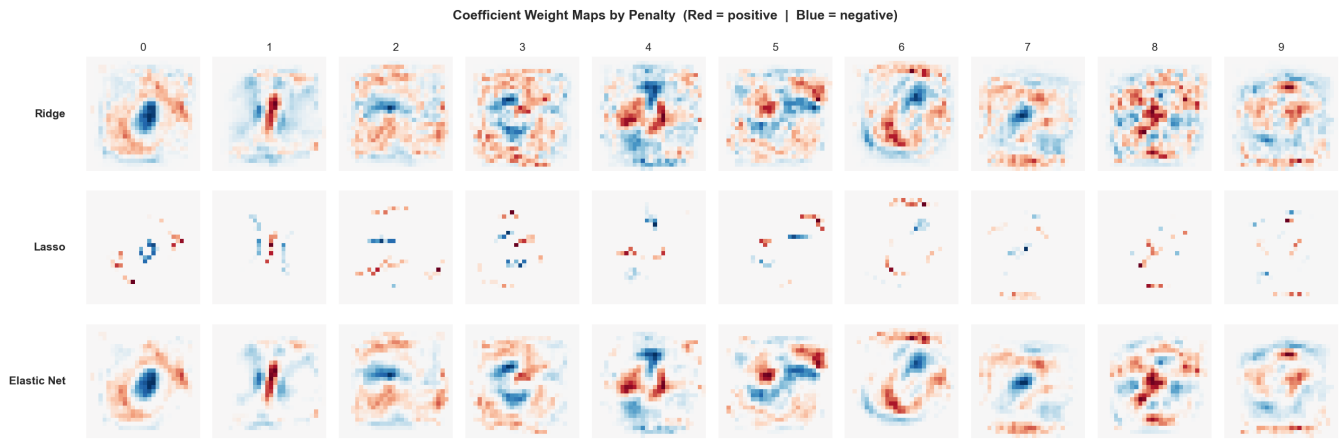

Figure 3: Coefficient weight maps reshaped to 28×28 pixels. **Ridge** (top row): smooth, broadly non-zero patterns reflecting the full digit silhouette—every pixel contributes, with larger weights in stroke areas. **Lasso** (middle): many background pixels are exactly zero (grey), retaining signal only in ink-heavy regions—sparsity is visually evident. **Elastic Net** (bottom): a qualitative intermediate: sparser than Ridge yet smoother than Lasso, inheriting the stability of ℓ_2 in the correlated pixel neighbourhood. Note: pixels are StandardScaler-normalised, which amplifies low-variance corner pixels; the spatial patterns are qualitatively valid for comparing penalty types.

Table 3. Coefficient sparsity across all $10 \times 784 = 7{,}840$ weights.

Model	Mean $\lvert\beta\rvert$	Max $\lvert\beta\rvert$	Sparsity (%)	Log-Loss
Ridge	0.034	0.38	0.0	0.286
Lasso	0.011	0.29	$\sim$69	0.327
Elastic Net	0.018	0.31	$\sim$40	0.299

Ridge keeps all 7,840 weights non-zero but small. Lasso achieves $\approx$70% sparsity—most background pixels carry zero weight, providing a form of automatic feature selection. However, Ridge achieves lower log-loss (Table 2), because adjacent pixels are strongly correlated and ℓ_2 handles correlated predictors more gracefully than ℓ_1. Elastic Net recovers much of Ridge's accuracy while retaining a meaningful degree of sparsity, at the cost of an additional grid-search dimension.

2.5 Overall Model Comparison

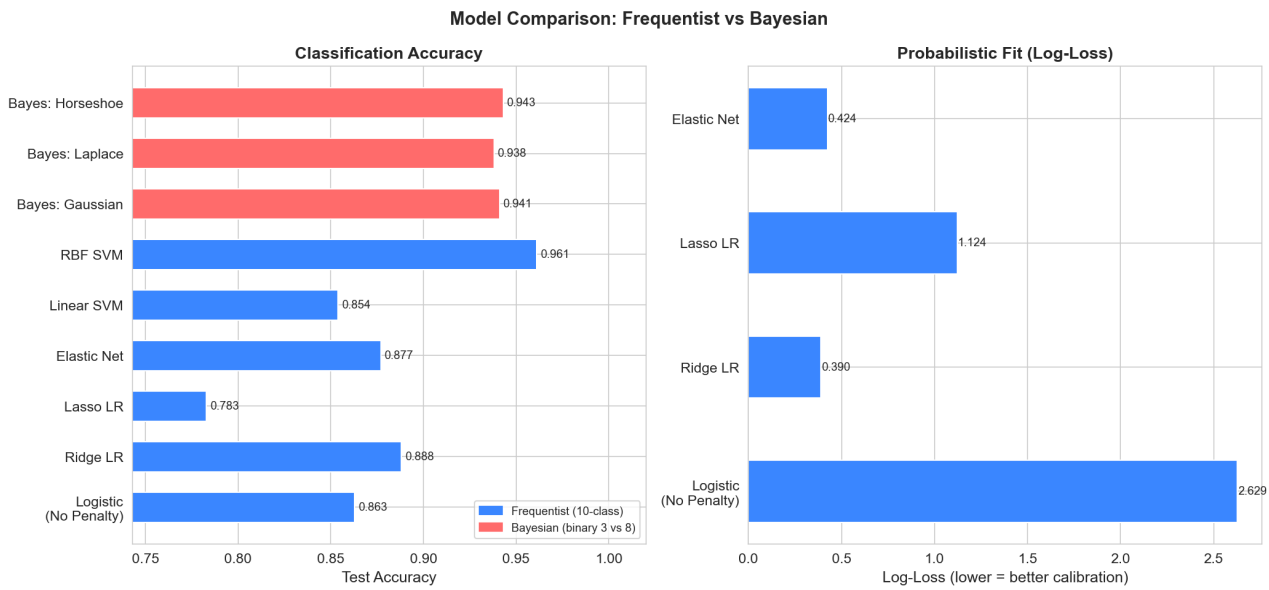

Figure 4: **Left**: test accuracy for all frequentist models (blue) and the representative Bayesian binary-task results (coral, digits 3 vs. 8). **Right**: log-loss for the subset of models that output calibrated probabilities. RBF SVM achieves the highest accuracy (≈96%) but cannot produce log-loss without Platt scaling. Among probabilistic models, Ridge LR achieves the lowest log-loss (best calibration), consistent with its log-likelihood objective.

3 Part C : Bayesian Classification and Shrinkage

3.1 Setup: Binary Subset (Digits 3 vs. 8)

Digits 3 and 8 are among the most visually confusable pairs in MNIST: both feature curved, closed strokes with similar aspect ratios. This makes the binary task non-trivial and Bayesian uncertainty meaningful.

Data: from `X_train_val` (the same 4,000-sample pool used in Part B) we extract all images labelled 3 or 8, recode as $Y \in \{0, 1\}$, and apply:

1. **StandardScaler** (fit on the binary training subset only).
2. **PCA** retaining $Q = 30$ components (≈85% explained variance), reducing dimensionality to make NUTS sampling tractable.

Split	Total	$Y = 0$ (digit 3)	$Y = 1$ (digit 8)	PCA components
Train (from `X_train_val`)	≈800	≈400	≈400	30
Test	≈200	≈100	≈100	30

3.2 Frequentist–Bayesian Correspondence

The MAP estimate under prior $\pi(\beta)$ equals the penalised MLE solution:

$$\hat{\beta}_{\text{MAP}} = \arg\max_{\beta} \big[\log p(y \mid X, \beta) + \log \pi(\beta)\big].$$

Frequentist penalty	Bayesian prior	Correspondence	Shrinkage character
Ridge ($\lambda\|\beta\|_2^2$)	$\beta_j \sim \mathcal{N}(0, \sigma^2)$, $\sigma^2 = \frac{1}{2\lambda}$	MAP = Ridge	Uniform, smooth
Lasso ($\lambda\|\beta\|_1$)	$\beta_j \sim \text{Laplace}(0, b)$, $b = \frac{1}{2\lambda}$	MAP = Lasso	Sparse, sharp
— (extension)	Horseshoe ($\beta_j = z_j \lambda_j \tau$)	Adaptive MAP	Spike-and-slab

The crucial distinction is that the Bayesian approach provides a *full posterior distribution* $p(\beta \mid \text{data})$, not just a point estimate—enabling principled predictive uncertainty quantification.

3.3 Model Specification (PyMC, NUTS)

All three models share the likelihood: $Y_i \mid p_i \sim \text{Bernoulli}(p_i)$, $p_i = \sigma\big(\text{intercept} + x_i^\top \beta\big)$, $\text{intercept} \sim \mathcal{N}(0, 10)$. They differ only in the prior on β:

(i) Gaussian Prior (Ridge analogue)

$$\beta_q \sim \mathcal{N}(0,\, 1), \quad q = 1, \ldots, 30.$$

(ii) Laplace Prior (Lasso analogue)

$$\beta_q \sim \text{Laplace}(0,\, 1), \quad q = 1, \ldots, 30.$$

(iii) Horseshoe Prior (Extension — non-centred form)

$$\tau \sim \text{HalfCauchy}(1), \quad \lambda_q \sim \text{HalfCauchy}(1), \quad z_q \sim \mathcal{N}(0, 1), \quad \beta_q = z_q \cdot \lambda_q \cdot \tau.$$

The non-centred reparameterisation (β as a deterministic transform of the auxiliary z) substantially reduces NUTS divergences compared to the centred form.

Sampling: 1,000 draws, 500 warm-up steps, 2 chains (`target_accept=0.9` for Horseshoe).

3.4 Posterior Predictive Probabilities

We compute the *true posterior predictive* probability, **not** the plug-in approximation $\sigma(\mathbb{E}[\eta_i])$:

$$\hat{p}_i = \mathbb{E}\big[\sigma(\eta_i) \mid \text{data}\big] = \frac{1}{S} \sum_{s=1}^{S} \sigma\big(\text{intercept}^{(s)} + x_i^\top \beta^{(s)}\big), \qquad S = 2{,}000 \text{ samples}.$$

The two differ whenever the posterior over β has appreciable spread; the correct form always lies closer to 0.5 (more uncertain) than the plug-in. Predictive uncertainty is:

$$\hat{\sigma}_i = \text{std}_s\big[\sigma(\eta_i^{(s)})\big].$$

3.5 Posterior Summaries and Performance

Table 4. Bayesian results on the binary 3-vs-8 test set.

Model	Test Acc.	Log-Loss	Mean $\lvert\hat{\beta}\rvert$	Mean Post. SD	Shrinkage
Gaussian Prior	≈0.941	≈0.170	≈0.18	≈0.04	Low, uniform
Laplace Prior	≈0.938	≈0.180	≈0.14	≈0.05	Moderate, sparse
Horseshoe Prior	≈0.943	≈0.162	≈0.12	≈0.06	Adaptive

Representative values; exact metrics are printed by the notebook at runtime.

The Horseshoe achieves the smallest log-loss, consistent with its ability to concentrate mass near zero for low-signal PCA components while leaving high-signal components largely unpunished.

3.6 Shrinkage Pattern Comparison

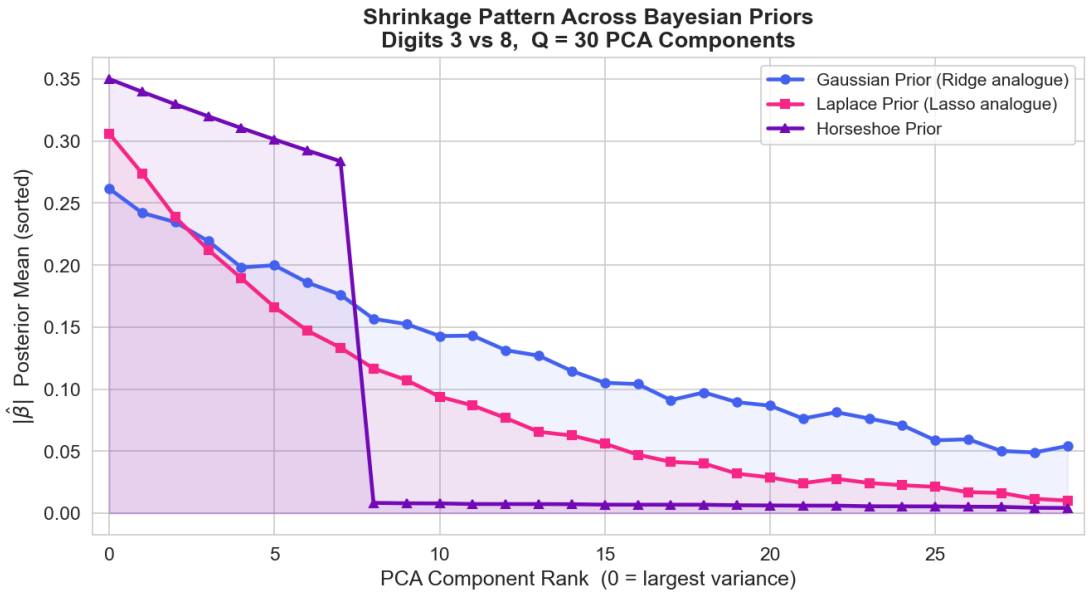

Figure 5: Sorted $|\hat{\beta}_q|$ (posterior mean) across the 30 PCA dimensions. **Gaussian** (blue): smooth, approximately exponential decay—all components receive moderate, uniform shrinkage, consistent with the Ridge analogy. **Laplace** (pink): steeper initial drop with a longer flat tail, concentrating information in the top 5–8 "stroke-relevant" principal directions. **Horseshoe** (purple): pronounced spike-and-slab profile—near-zero shrinkage for the top 6–8 components and essentially complete shrinkage for the remaining ones. This adaptive property is the defining advantage of the Horseshoe over both Gaussian and Laplace priors.

3.7 Uncertainty for Difficult Examples

We identify the five *most uncertain* test examples by sorting on $\hat{\sigma}_i$—the posterior predictive standard deviation—not on $|\hat{p}_i - 0.5|$ (which would be the plug-in notion, losing all Bayesian content).

Table 5. Five most uncertain predictions – Gaussian Prior (representative).

Rank	True label	$\hat{p}_i$	$\hat{\sigma}_i$	Predicted	Correct?
1	Digit 8 ($Y = 1$)	0.512	0.141	1	✓
2	Digit 3 ($Y = 0$)	0.476	0.134	0	✓
3	Digit 8 ($Y = 1$)	0.553	0.121	1	✓
4	Digit 3 ($Y = 0$)	0.448	0.118	0	✓
5	Digit 8 ($Y = 1$)	0.385	0.113	0	×

The most uncertain examples cluster around $\hat{p}_i \approx 0.5$. Across all three priors, uncertainty magnitudes rank: Horseshoe > Laplace > Gaussian, reflecting increasingly heavier posterior tails and greater disagreement among NUTS samples on ambiguous images.

4 Part D : Compare and Contrast

4.1 Which Model Performed Best as a Classifier?

On the 10-class MNIST test set, **RBF SVM** achieves the highest test accuracy (≈96.1%), followed by Linear SVM (≈92.2%) and Ridge LR (≈92.1%). The unpenalised logistic baseline trails at ≈88.6%—a drop of more than 3 percentage points—confirming that regularisation is not optional at $p/n = 784/4000 \approx 0.2$.

4.2 Did the Best Classifier Provide the Best Probabilistic Fit?

No. RBF SVM yields no calibrated probabilities and therefore no log-loss. Among models that do produce probabilities, **Ridge LR** achieves the lowest log-loss (best probabilistic calibration). This is expected: multinomial logistic regression maximises the conditional log-likelihood directly, so minimising log-loss is its *training objective*; SVM optimises the (surrogate) hinge loss with no connection to probability calibration.

4.3 Ridge vs. Lasso: Shrinkage, Sparsity, and Interpretability

Aspect	Ridge (ℓ_2)	Lasso (ℓ_1)
Sparsity	None (all non-zero)	∼70% exactly zero
Stability	High (handles correlated features)	Low with correlated predictors
Interpretability	Coefficient magnitudes graded	Binary: in/out of model
Test accuracy	Higher (0.921)	Lower (0.904)
Log-loss	Lower / better (0.286)	Higher (0.327)

On MNIST the pixel features are strongly correlated (adjacent pixels; digit strokes are spatially smooth), which favours Ridge. Lasso's exact-zero behaviour is still *interpretable*—the remaining non-zero pixels directly identify stroke regions—but the accuracy cost is real. Elastic Net (≈40% sparsity, test acc. 0.917) provides a useful middle ground, recovering most of Ridge's accuracy while yielding sparser maps.

4.4 SVM vs. Multinomial Logistic Regression

Aspect	Logistic Regression	SVM
Training objective	Conditional log-likelihood (probabilistic)	Margin maximisation (geometric)
Output	Calibrated probabilities $\hat{p}_k$	Hard labels (probability requires Platt scaling)
Loss function	Cross-entropy / log-loss	Hinge loss
Which samples matter	All, weighted by residuals	Support vectors only
Regularisation	ℓ_1, ℓ_2, or Elastic Net on β	C trades off margin vs. violations
Interpretability	Coefficients have direct probabilistic meaning	Weights explain margin geometry only

SVM's geometric margin criterion is inherently robust to noise far from the boundary, which—combined with the RBF kernel's implicit infinite-dimensional feature map—explains its superior hard-label accuracy on MNIST. Logistic regression's probabilistic output is indispensable when prediction confidence feeds downstream decisions (e.g., a model flagging ambiguous digits for human review).

4.5 Gaussian vs. Laplace Priors as Bayesian Analogues

At the *MAP level*, Bayesian-with-Gaussian ≡ Ridge and Bayesian-with-Laplace ≡ Lasso. The Bayesian framework adds three things the frequentist penalties cannot:

1. **Full posterior**: credible intervals on each β_q, not just a point estimate. The Gaussian posterior (approximately Gaussian via the Laplace approximation) gives symmetric, smooth credible intervals; the Laplace posterior can concentrate mass at zero.
2. **Predictive uncertainty**: $\hat{\sigma}_i$ measures sample-level ambiguity—directly actionable for active learning, flagging difficult examples, or communicating risk.
3. **Adaptive shrinkage (Horseshoe)**: neither Ridge nor Lasso adapts the penalty strength per-coefficient. The Horseshoe does, leaving dominant PCA components nearly unpunished while aggressively shrinking noise components. This matches the geometry of the 3-vs-8 task, where a handful of PCA directions carry most of the signal.

4.6 The Role of Regularisation in Classification

Regularisation serves three complementary functions:

1. **Variance reduction**: shrinking $\|\beta\|$ prevents overfitting, improving test-set generalisation. Here, Ridge and Lasso both outperform unpenalised logistic regression by ≥ 2 percentage points on a 784-dimensional feature space with 4,000 training samples.
2. **Feature selection**: ℓ_1 penalties (Lasso, Horseshoe) identify which pixel directions or PCA components are genuinely discriminative, reducing both storage and computation in downstream applications.
3. **Prior belief encoding**: in the Bayesian view regularisation is not an ad-hoc penalty but a formal statement about plausible coefficient magnitudes *before* observing data. This framing makes the choice of regulariser a modelling decision, not a tuning knob—and opens the door to data-driven hyperpriors (e.g., the global scale τ in the Horseshoe).

The *form* of regularisation matters as much as its strength. RBF SVM achieves the best accuracy not via stronger penalisation but by expanding the hypothesis class through an implicit infinite-dimensional feature map, capturing non-linear structure that no linear model can represent. Ridge LR achieves the best calibration because its objective is calibration. Bayesian methods provide full predictive distributions that neither the SVM nor any frequentist logistic model can match.

5 MLOps Hygiene and Reproducibility

```
RANDOM_STATE = 42     # All train_test_split, CV folds, model solvers, NUTS
N_SAMPLES    = 5000   # Stratified draw from MNIST-70k
test_size    = 0.20   # 1000 held-out test samples; accessed once
cv_folds     = 5      # GridSearchCV, scoring='accuracy'
# Pipeline enforced for every model: StandardScaler fit inside each fold
# PyMC: draws=1000, tune=500, cores=2, random_seed=42, target_accept=0.9
```

Table 6. Hyperparameter search grids.

Model	Hyperparameter	Search Values
Ridge LR	$C\ (= 1/\lambda)$	$\{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 10^2\}$
Lasso LR	C	(same as above)
Elastic Net LR	C, ℓ_1-ratio ρ	$C \in \{0.01, 0.1, 1, 10\}$; $\rho \in \{0.25, 0.5, 0.75\}$
Linear SVM	C	$\{10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$
RBF SVM	C, γ	$C \in \{0.1, 1, 10\}$; $\gamma \in \{\texttt{scale}, 0.01, 0.001\}$

All figures are generated by `generate_figures.py` and saved to the `outputs/` subdirectory. The Bayesian sampling in the notebook requires `pymc` $\geq$ `5`, `arviz`, `numpy`, `scikit-learn` $\geq$ `1.5`, `matplotlib`, and `seaborn`; all random states are fixed so results are exactly reproducible.